



Project Documentation.

An AI-powered vision and robotics platform for real-time automated waste sorting, built around a live industrial monitoring dashboard.

COMPUTER VISION

YOLO OBJECT DETECTION

ROBOTICS INTEGRATION

INDUSTRIAL DASHBOARD

PROJECT NAME	Trace Sort AI
PROJECT DOMAIN	Smart Waste Sorting Robot & Industrial AI Monitoring System
TEAM MEMBERS	Abbas Lotfi • Ara Tajaddini • Reza Esmaeili • Sina Niknejad
DEPLOYMENT TARGETS	Windows Standalone Setup Installer & macOS Native Application
DOCUMENT TYPE	Technical Overview & Installation / Usage Guide

Table of Contents

01 Executive Summary

What TraceSort AI is, the problem it solves, and how the system fits together.

02 System Architecture

End-to-end data flow from camera input to robotic actuation.

03 AI & Computer Vision Pipeline

Detection model, training configuration, and the 18 → 5 category classification scheme.

04 Robotics & Hardware Integration

Kinematics engine, Arduino serial protocol, and the robot command payload.

05 The TraceSort AI Dashboard

A guided tour of every panel, metric, and control on the live dashboard.

06 Innovations & Technical Creativity

Eight design decisions that set TraceSort AI's pipeline apart.

07 Engineering Challenges & Solutions

Five real problems encountered during development, and how each was resolved.

08 Installation Guide — Windows

Step-by-step setup using the Windows standalone installer.

09 Installation Guide — macOS

Step-by-step setup using the native macOS application.

10 Using the Dashboard

Day-to-day operation: starting a session, monitoring, and handling alerts.

11 Configuration Reference

Every tunable parameter in `config.yaml` explained.

12 Vision & Future Roadmap

The eleven-point plan guiding TraceSort AI from prototype to production fleet.

Executive Summary

The vision behind TraceSort AI, in brief

TraceSort AI is an AI-powered, camera-driven waste sorting system that identifies recyclable and non-recyclable materials on a moving conveyor belt in real time, and issues precise pick-and-place instructions to a robotic arm. The system pairs a custom-trained object detection model with a live industrial monitoring dashboard, giving operators full visibility into sorting throughput, accuracy, bin capacity, and equipment health at all times.

At its core, TraceSort AI answers a simple operational question — *"what is on the belt, where exactly is it, and where should it go?"* — and answers it dozens of times per second, translating a video frame into a physical robot command in milliseconds.

In one sentence: TraceSort AI watches a conveyor belt, recognizes what kind of waste is passing by, calculates exactly where and how a robotic arm should grip it, and gives plant operators a real-time dashboard to supervise, intervene, and measure performance.

Core Capabilities

CAPABILITY	DESCRIPTION
Real-time detection & tracking	Identifies and tracks waste items frame-by-frame using a custom YOLO-based model with persistent object IDs.
Material classification	Groups 18 fine-grained litter types into five actionable sorting categories: Glass, Metal, Paper, Plastic, and Waste.
Robotic kinematics	Converts pixel coordinates into real-world millimeter coordinates and computes grip angle, timing, and required gripping force.
Hardware actuation	Streams structured JSON pick commands to an Arduino-controlled robotic arm over a serial connection.
Live operations dashboard	A modern, dark-themed industrial UI displaying throughput, accuracy, OEE, revenue recovery, bin status, and system logs.
Safety & resilience	Emergency stop, automatic conveyor halting on bin overflow, and graceful offline operation if hardware is disconnected.
Model benchmarking	One-click, in-dashboard evaluation of detection accuracy (mAP, precision, recall) against a held-out validation set.

System Architecture

From camera frame to robotic action

TraceSort AI is organized as a closed-loop pipeline. Each stage consumes the output of the previous one, running continuously while the dashboard is active:

- 1

Video Capture

A frame is captured either from a live webcam feed (streamed continuously) or from an uploaded video file (processed frame-by-frame, resized and padded to a fixed 640×640 working resolution).
- 2

Detection & Tracking

The frame is passed to the trained YOLO model, running with ByteTrack persistence enabled so that each physical object keeps a stable track ID across frames instead of being re-detected from scratch every cycle.
- 3

Category Mapping

Each detection's raw label (one of 18 fine-grained litter classes) is mapped to one of five operational sorting categories — Glass, Metal, Paper, Plastic, or Waste — using a fixed lookup table.
- 4

Trigger-Line Evaluation

A virtual "trigger line" is drawn across the belt at a configurable position. Objects are only queued for picking once their center crosses this line, and each track ID is picked at most once, preventing duplicate commands for the same physical item.
- 5

Priority Selection

When multiple objects reach the trigger line together, the system selects the next target using an environmental priority order (Metal, then Plastic, then Glass, then Paper, then Waste), breaking ties by detection confidence.
- 6

Kinematics & Payload Generation

Pixel coordinates are converted into real-world millimeter coordinates, the object's orientation angle is measured, and a structured JSON pick command is generated with position, angle, timing, and gripping force.
- 7

Hardware Transmission

The JSON payload is transmitted over a serial (USB) connection to the Arduino-controlled robotic arm, which executes the physical pick-and-place action.
- 8

Dashboard Feedback

Every detection, decision, and hardware event is reflected live on the operator dashboard — annotated video, metrics, bin levels, and system logs — closing the loop between the physical process and the operator.

Technology Stack

LAYER	TECHNOLOGY
Object detection & tracking	Ultralytics YOLO with ByteTrack multi-object tracking
Computer vision utilities	OpenCV (contour analysis, orientation estimation, frame annotation)
Numerical & tabular processing	NumPy, Pandas
ML runtime	PyTorch, with automatic GPU (CUDA) acceleration and safe CPU fallback
Dashboard & UI framework	Gradio (Blocks API) with a fully custom industrial-themed CSS layer
Hardware communication	PySerial, via a dedicated Arduino connection module with auto port discovery
Desktop packaging	Native desktop windowing layer for a standalone, browser-free application experience

AI & Computer Vision Pipeline

How TraceSort AI learned to see waste

Model & Training Configuration

The detection model is trained using the Ultralytics YOLO framework, starting from a lightweight pretrained checkpoint and fine-tuned on a custom waste-detection dataset with 18 annotated litter classes. The training run is configured for strong generalization under real-world belt conditions — variable lighting, occlusion, and object orientation — through an extensive augmentation policy.

PARAMETER	VALUE
Base checkpoint	YOLO11n (nano variant, optimized for speed)
Epochs	100 (with early-stopping patience of 15)
Image size	640 × 640
Batch size	16
Training device	GPU-accelerated (CUDA)
Mosaic augmentation	1.0
Mixup augmentation	0.15
Copy-paste augmentation	0.10
Rotation / translation / scale	±15° / 0.10 / 0.50
Shear / perspective	2.0 / 0.0005
Horizontal / vertical flip	0.50 / 0.20
HSV color jitter (H / S / V)	0.015 / 0.70 / 0.40
Random erasing	0.40

From 18 Raw Classes to 5 Sorting Categories

The underlying dataset is annotated at a fine-grained level (e.g. "Bottle cap", "Broken glass", "Cigarette"). For operational sorting, TraceSort AI collapses these into five actionable categories that map directly to physical bins and material recovery value:

SORTING CATEGORY	RAW CLASSES MAPPED IN
Metal	Aluminium foil, Can, Pop tab
Plastic	Bottle cap, Bottle, Lid, Other plastic, Plastic bag / wrapper, Plastic container, Straw
Glass	Broken glass
Paper	Carton, Cup, Paper
Waste	Cigarette, Other litter, Styrofoam piece, Unlabeled litter

Sorting Priority & Recovery Value

When several items arrive at the pick point simultaneously, TraceSort AI resolves the queue using an environmental priority order combined with a per-item estimated recovery value used for the dashboard's revenue tracking:

CATEGORY	SORTING PRIORITY	ESTIMATED RECOVERY VALUE
Metal	1 (highest)	\$0.08 / item
Plastic	2	\$0.05 / item
Glass	3	\$0.04 / item
Paper	4	\$0.03 / item
Waste	5 (lowest)	\$0.00 / item

Continuous Benchmarking

The dashboard includes a built-in **Run Benchmark** action that evaluates the currently loaded model against a held-out validation set, automatically retrieving the dataset if it is not already present locally. The benchmark reports mAP@50, precision, and recall, giving operators and engineers a quick, repeatable way to confirm detection quality without leaving the dashboard.

Robotics & Hardware Integration

Turning a detection into a physical pick

Kinematics Engine

Once an object crosses the trigger line, TraceSort AI converts its pixel position into real-world coordinates the robotic arm can act on. The engine calculates:

- **World X position** — the object's horizontal offset from the belt's centerline, converted from pixels to millimeters via a calibrated scale factor.
- **Remaining distance to the grasp zone** — how far the object still has to travel before it reaches the arm's optimal picking position.
- **Time-to-grab** — the estimated time, in milliseconds, until the object reaches the grasp zone, derived from the belt's configured speed.
- **Orientation angle** — the object's rotation, measured directly from its visual contour using a minimum-area bounding rectangle, so the gripper can align to it correctly.
- **Required gripping force** — looked up per material category, so fragile glass and rigid metal are handled with different amounts of force.

Robot Command Payload

Each pick decision is serialized into a compact JSON message and transmitted to the robot controller. This is the single contract between the vision system and the physical hardware:

FIELD	MEANING
cmd	Command type (e.g. PICK)
cls	Sorting category of the target object
x	Horizontal world coordinate, in millimeters
y	Remaining distance to the grasp zone, in millimeters
z	Fixed vertical approach height, in millimeters
force	Target gripping force, in Newtons
theta	Object orientation angle, in degrees
ttg_ms	Time-to-grab, in milliseconds
ts	Unix timestamp of the command

Arduino Communication Layer

TraceSort AI communicates with the robot controller over a serial (USB) connection. On startup, it attempts to connect to a default port and, if unavailable, automatically scans all detected serial ports until a working connection is established. If no hardware is found, the system continues to run in **offline mode** — the full vision pipeline, dashboard, and metrics remain active, with pick commands simply logged instead of transmitted. A **Reset Arduino** control in the dashboard allows the operator to safely close and re-establish the hardware connection at any time, without restarting the application.

Conveyor Safety Interlocks

The system continuously monitors simulated bin fill levels against configured capacities. The moment any bin reaches capacity, TraceSort AI automatically issues a conveyor-stop command over serial, halts new pick decisions, and displays a clear on-screen alert — preventing overflow until an operator empties the affected bin from the dashboard, at which point the conveyor resumes automatically.

The TraceSort AI Dashboard

A guided tour of the operator interface

The dashboard is TraceSort AI's command center — a dark, glass-panelled industrial interface designed for at-a-glance monitoring on a plant floor. It is organized into clearly separated zones:

Live Visualization Panel

The primary panel streams the annotated camera or video feed, with bounding boxes, track IDs, category labels, confidence scores, and the trigger line drawn directly on the video. Operators can switch the input source between a **live webcam feed** and an **uploaded video file**, and trigger a model **benchmark** or an Arduino **reconnect** directly from this panel.

Real-Time Metrics Column

Metric	What It Shows
Cycle Velocity	The current per-frame processing cycle time.
Motion Accuracy	The rolling average detection confidence across all sorted items.
System OEE	Overall Equipment Effectiveness — a composite score combining availability, performance against the ideal cycle time, and detection quality.
Sorting Speed	Items sorted per minute, measured over a rolling 60-second window.
Total Revenue	Estimated cumulative material recovery value, based on category-specific recovery rates.

Each metric carries a live status indicator — **Optimal / Nominal**, **Stable**, or **Alert** — so deviations are visible without reading exact numbers.

Performance Analytics

A live line chart plots total items sorted over time, giving an immediate visual read on throughput trends across the current session.

Bin Status & Management

A live table tracks sorted counts, share of total volume, and fill status for each of the five categories. Operators can select any bin and use the **Empty Bin** control to reset its fill level once it has been physically emptied — historical totals are preserved, only the physical capacity counter resets.

System Logs & Production Core

A continuously updating event log records every sorting decision, hardware event, and system state change with a timestamp. Alongside it, the **Production Core** panel displays the full detail of the most recent robot command, and a dedicated **Arduino Log** shows the raw JSON payloads transmitted to the hardware — invaluable for debugging and hardware integration work.

Emergency & Safety Controls

A dedicated control row provides three operator-facing safety actions:

- Emergency Stop** — immediately halts all detection, sorting, and conveyor operation, and transmits an emergency-stop command to the hardware.
- Resume** — releases the emergency stop and re-evaluates system state (the conveyor will only restart automatically if no bins remain full).
- Reset Metrics** — clears session counters, bin levels, and downtime tracking to start a fresh monitoring session.

Innovations & Technical Creativity

Design decisions that set TraceSort AI's pipeline apart

Beyond wiring a detector to a robot arm, TraceSort AI embeds a number of deliberate engineering choices that make the system practical, safe, and economically meaningful on a real production line. The eight highlights below are the ones the team considers most distinctive.

1 Two-Stage Class Mapping (18-to-5 Logic)

Converts the 18 raw TACO dataset classes into five operational sorting categories (Glass, Metal, Paper, Plastic, Waste), so that gripper force selection, bin routing, and economic-value calculation can all be driven by one simplified, actionable label.

2 Priority-Based Sorting Decision Logic

When multiple objects reach the trigger line at the same time, the system doesn't just pick the first one it sees — it selects the most valuable target using environmental/economic priority (Metal > Plastic > Glass > Paper > Waste), with detection confidence as the tiebreaker.

3 Angle & Kinematics Estimation from a Clean Contour Copy

Before calculating orientation, the system takes a copy of the frame without bounding-box overlays, extracts the object's contour from that clean copy, and computes the gripper rotation angle (θ), real-world millimeter coordinates (X, Y, Z), and the remaining time-to-grab (TTG) — keeping the angle measurement free of annotation artifacts.

4 Dynamic Grip Force Adjustment

Gripper force, in Newtons, is selected according to material type rather than applied uniformly — for example, roughly 20N for glass to avoid breakage versus roughly 85N for paper — so fragile and rigid materials are handled appropriately.

5 Lightweight, GPU-Independent Architecture (YOLO11n)

The Nano model variant was deliberately chosen so the system runs smoothly on an ordinary laptop CPU, keeps the installer lightweight, and still delivers a high frame rate on typical evaluation hardware — GPU acceleration is a bonus, not a requirement.

6 Native Desktop Architecture via PyWebview

Combines Gradio for the interface layer with a native desktop windowing shell, so TraceSort AI runs as a self-contained desktop application — no separate browser window to manage and no Python environment for the end user to install.

7 Built-in Benchmark Tool

Automatically downloads the evaluation dataset from GitHub when it isn't already present, then computes mAP@50, Precision, and Recall directly inside the dashboard — no separate scripts or manual dataset wrangling required.

8 Industrial KPI Modeling (OEE)

Computes genuine production-line rates — Availability, Performance, Quality, and the composite OEE score — alongside a live economic-value calculation of sorted waste, turning raw detection counts into operations-grade metrics.

Engineering Challenges & Solutions

Real problems encountered during development, and how each was resolved

Building a vision-to-robot pipeline that runs continuously and safely surfaced a number of concrete engineering problems. Each one below was identified during development and addressed directly in the codebase.

1. OpenCV Version Inconsistency in Angle Calculation

Challenge The angle convention used by `cv2.minAreaRect` changed starting in OpenCV 4.5, which caused incorrect gripper rotation angles depending on which OpenCV version was installed.

Solution The application automatically detects the installed OpenCV version at startup and applies the matching angle convention — modern or legacy — so gripper orientation stays correct regardless of environment.

2. Memory Growth During Long Monitoring Sessions

Challenge The ByteTrack algorithm continuously assigns new track IDs over time, which gradually filled up memory during extended monitoring sessions.

Solution A dedicated cleanup routine discards any track ID that hasn't been seen for more than 30 seconds, keeping memory usage bounded during long shifts.

3. Preventing Duplicate Sorting of the Same Object

Challenge An object lingering near the trigger line across several consecutive frames could be picked up multiple times, triggering duplicate robot commands for the same physical item.

Solution Every processed track ID is stored in a dedicated set, so subsequent frames for the same object are automatically skipped once it has already been queued for a pick.

4. Arduino Disconnection & Offline Robustness

Challenge If the robot controller was unplugged or otherwise unavailable, the application risked crashing outright instead of continuing to operate.

Solution Serial communication uses non-blocking connection handling with full exception management, a genuine offline mode that keeps the vision pipeline and dashboard running, and a manual Reconnect control so hardware can rejoin without a restart.

5. Bin Overflow Safety Interlock

Challenge If a bin reached capacity while the line kept running, newly sorted material had nowhere to go and risked spilling.

Solution Bin capacity is monitored continuously; the moment any bin fills, the system automatically sends a `STOP_CONVEYOR` command to the Arduino and tracks the resulting downtime as part of the OEE calculation.

Installation Guide — Windows

Setting up TraceSort AI on Windows

System Requirements

REQUIREMENT	DETAILS
Operating System	Windows 10 or later (64-bit)
Camera	A connected USB or built-in webcam for live detection
Hardware link (optional)	An Arduino-based robot controller connected via USB, for physical sorting; the dashboard runs fully in offline mode without it
Graphics	A CUDA-capable NVIDIA GPU is recommended for best performance; the system automatically falls back to CPU processing if none is found
Network	Only required on first run of the benchmark feature, to download the validation dataset

Setup Steps

1 Run the Windows installer

Locate the TraceSort AI Windows setup package provided with the project and double-click it to launch the installer.

2 Approve the security prompt

Windows SmartScreen may flag the installer as coming from an unrecognized publisher, since it is an internal project build. Select "**More info**" → "**Run anyway**" to proceed.

3 Complete the installation wizard

Follow the on-screen prompts, choose an install location, and allow the installer to finish. A TraceSort AI shortcut will be created on your Desktop and/or Start Menu.

4 Connect your hardware (optional)

If you are running a physical sorting cell, connect the Arduino robot controller to the PC via USB before launching the application, so it can be auto-detected on startup.

5 Launch TraceSort AI

Open TraceSort AI from its shortcut. The application starts a local dashboard server and automatically opens the TraceSort AI interface in its own application window — no manual browser navigation required.

6 Grant camera permissions

On first launch, allow TraceSort AI to access your webcam when prompted by Windows, so live detection can begin.

Tip: If the application window does not appear automatically, check that no other program is using port 8001 on your machine, and confirm the app is not blocked by your firewall or antivirus software.

Installation Guide — macOS

Setting up TraceSort AI on macOS

System Requirements

REQUIREMENT	DETAILS
Operating System	macOS 12 (Monterey) or later, Apple Silicon or Intel
Camera	A connected USB or built-in webcam for live detection
Hardware link (optional)	An Arduino-based robot controller connected via USB, for physical sorting; the dashboard runs fully in offline mode without it
Graphics	Runs on CPU by default on macOS; performance scales with the host Mac's processor
Network	Only required on first run of the benchmark feature, to download the validation dataset

Setup Steps

1 Open the macOS installer package

Locate the TraceSort AI macOS application package provided with the project and open it to begin installation.

2 Move the app to Applications

If presented with a standard macOS installer window, drag the TraceSort AI icon into your **Applications** folder.

3 Approve Gatekeeper

Since this is an internally built application, macOS Gatekeeper may block the first launch. **Right-click (or Control-click) the TraceSort AI app → "Open"**, then confirm in the dialog that appears. This is only required once. Alternatively, approve it under **System Settings → Privacy & Security → "Open Anyway."**

4 Connect your hardware (optional)

If running a physical sorting cell, connect the Arduino robot controller via USB before launching the app so it can be auto-detected on startup.

5 Launch TraceSort AI

Open TraceSort AI from your Applications folder or Launchpad. The application starts its local dashboard server and opens the TraceSort AI interface directly in its own window.

6 Grant camera and accessory permissions

Approve the macOS prompts for **Camera** access, and for USB accessory access if connecting an Arduino controller, so the full pipeline can operate.

Tip: If macOS reports the app as "damaged" or unable to be opened, this is typically a Gatekeeper quarantine flag on an unsigned internal build rather than an actual file issue — re-approving via **System Settings → Privacy & Security** resolves it.

Using the Dashboard

Day-to-day operation, once TraceSort AI is running

Starting a Live Session

With **Webcam** selected as the input source, TraceSort AI begins streaming and analyzing frames automatically — no start button is required. Detected items are annotated live, and every item that crosses the trigger line is logged, counted, and, if hardware is connected, sent to the robot arm.

Analyzing a Recorded Video

Switch the input source to **Video File**, upload a clip, and select **Start Processing**. TraceSort AI will play the video through the full detection pipeline exactly as it would a live feed, useful for testing, demonstrations, or reviewing footage from a shift.

Monitoring Performance

Keep an eye on the **Real-Time Metrics** column throughout a session. A shift from **Optimal** to **Stable** or **Alert** status on Motion Accuracy or System OEE is an early signal worth investigating — it may indicate lighting changes, a camera obstruction, or unusually heavy occlusion on the belt.

Responding to a Full Bin

When a bin reaches capacity, the conveyor halts automatically and the System Logs panel will clearly flag which bin triggered the stop. Physically empty the indicated bin, then select **Empty Bin** for that category in the dashboard — the conveyor will resume automatically once no bins remain full.

Using Emergency Stop

Select **EMERGENCY STOP** at any time to immediately freeze all detection, sorting, and conveyor motion. This is independent of bin-full logic and takes priority over every other system state. Once the situation is resolved, select **Resume** to re-enable operation.

Running a Benchmark

Select **Run Benchmark** from the main panel to evaluate the currently loaded detection model against the validation dataset. Results — mAP@50, precision, and recall — are printed to the System Logs panel. Note that live video streaming pauses briefly while the benchmark runs.

Reconnecting Hardware

If the robot controller is power-cycled or unplugged mid-session, select **Reset Arduino** to safely close and re-establish the serial connection without restarting the whole application.

Configuration Reference

Every tunable parameter in `config.yaml`

TraceSort AI is calibrated entirely through a single configuration file, allowing the system to be adapted to different physical belts, cameras, and robot arms without touching application code.

Hardware & Communication

KEY	PURPOSE
<code>hardware.default_port</code>	The serial port to try first when connecting to the Arduino controller (e.g. COM3 on Windows).
<code>hardware.baudrate</code>	Serial communication speed between the host application and the robot controller.

Conveyor Calibration

KEY	PURPOSE
<code>conveyor.speed_mm_s</code>	Physical belt speed in millimeters per second, used to calculate time-to-grab.
<code>conveyor.direction</code>	Direction of belt travel relative to the camera frame (upward or downward).
<code>conveyor.grasping_zone_y_mm</code>	The distance along the belt, in millimeters, that defines the arm's optimal grasp position.

Vision & Camera Calibration

KEY	PURPOSE
<code>vision.trigger_line_ratio</code>	Vertical position of the trigger line as a fraction of frame height.
<code>vision.trigger_tolerance_px</code>	Pixel tolerance window around the trigger line for registering a pick event.
<code>vision.scale_factor_mm</code>	Conversion factor from image pixels to real-world millimeters.

Bin & Force Configuration

KEY	PURPOSE
<code>bin_capacities</code>	Maximum item count per bin before the system automatically halts the conveyor.
<code>grip_forces</code>	Target gripping force, in Newtons, defined individually for Glass, Paper, Plastic, Metal, and Waste.

Production & OEE Constants

KEY	PURPOSE
<code>PLANNED_PRODUCTION_TIME</code>	Reference planned run duration used in OEE-style reporting.
<code>IDEAL_CYCLE_TIME</code>	The target, best-case time to sort a single item — the benchmark used to score performance efficiency.
<code>zm</code>	Fixed vertical (Z-axis) approach offset sent to the robot arm for every pick.

Note: Changes to `config.yaml` take effect the next time TraceSort AI is launched. Always re-calibrate `scale_factor_mm` and `trigger_line_ratio` after physically repositioning the camera.

Vision & Future Roadmap

From working prototype to a production sorting fleet

TraceSort AI's current release proves the full loop — detection, decision-making, and hardware actuation — end to end. The roadmap below outlines how the system will mature from a single-line prototype into a scalable, self-improving, multi-facility platform.

1 Robotic Integration

Integrate a physical robotic arm that consumes the existing serial payload to perform real picks.

2 Hardware Calibration

Field-calibrate conveyor speed, bin capacities, and cycle-time constants against real physical hardware.

3 Secondary Actuation

Add a second actuator for automated bin evacuation using the same output protocol.

4 Next-Gen Model Migration (YOLO26 Upgrade)

Fine-tune on the emerging YOLO26-OBb (oriented bounding box) architecture for further reduced latency, higher mAP under heavy occlusion, more precise angle estimation for irregularly shaped items, and a lower memory footprint.

5 Human-in-the-Loop MLOps & Active Learning

Automatic caching of low-confidence detections (confidence below 0.55) during shifts; an Operator Review Panel inside the dashboard for one-click label corrections; automated generation of YOLO-format annotations for continuous model fine-tuning — a self-improving system.

6 Physical Hardware & Sensor Expansion

Integration of ultrasonic bin sensors for continuous physical depth and volume measurement; multi-arm routing with dynamic target allocation distributing work across multiple robotic arms on high-speed lines.

7 Long-Term Business Intelligence (BI) Analytics

SQLite / PostgreSQL historical database integration; monthly executive dashboards tracking overall recovery tonnage, OEE trends, and financial yield.

8 Edge Hardware Deployment

Porting the vision model from the host PC to embedded edge AI platforms (e.g. NVIDIA Jetson Orin Nano, or Raspberry Pi 5 with a Hailo-8 AI accelerator) to drastically reduce power consumption and deployment cost.

9 Model Optimization (TensorRT / ONNX)

Converting the trained YOLO26 model to TensorRT and ONNX formats to achieve ultra-low latency and maximum FPS on edge hardware.

10 Cloud Analytics & Fleet Management

A centralized cloud dashboard to monitor multiple waste-sorting lines across facilities in real time, aggregating data for executive reporting and operational insight — alongside continuous future innovation guided by industry needs.

11 LLM-Driven Circular Economy Analytics

An intelligent analytics module that uses large language models to analyze end-of-shift recycling data, estimate the added value of collected materials, and generate industrial recommendations for upcycled product opportunities.



TRACESORT AI

Built to sort smarter. Engineered to scale.

TraceSort AI turns a standard camera and a conveyor belt into an intelligent, measurable, and continuously improving material recovery system.

TEAM

Abbas Lotfi • Ara Tajaddini • Reza Esmaeili • Sina Niknejad

DOCUMENT

TraceSort AI — Technical Documentation